Fire-Fit: A Smart-TV Personal Trainer That Uses the Viewer's Phone as a Private Pose and Voice Sensor

[Author Name]
[Department, Institution]
[City, Country]
[e-mail]

***Abstract*—Smart TVs are the largest screen in most homes and a natural place to exercise, but they have no camera or microphone that apps can use, so TV workout apps cannot count repetitions, check form or respond to the user. We present Fire-Fit, a personal trainer for Amazon Fire TV and Vega OS in which the TV runs the session and the viewer's own phone, opened from a QR code, acts only as its sensor. The phone runs MediaPipe pose estimation in the browser, so video never leaves it; only rep events, form cues and recognised voice commands are relayed to the TV. A large language model (Amazon Nova 2 Lite on Amazon Bedrock) generates each workout as schema-validated structured output. The core of the system is a rule-based rep counter for 19 exercise families with a set of robustness mechanisms: 3D world landmarks, aspect-ratio correction, frame and posture gating, side selection, a start-position rule, minimum rep durations, alternation for bilateral moves and One Euro smoothing. On a synthetic benchmark of 246 sequences built from the app's 23 counted exercise animations under six camera conditions, the full counter reaches a macro-averaged mean absolute error (MAE) of 0.063 and an off-by-one accuracy (OBOA) of 92.1% with no phantom reps on distractor motion, against 0.212, 78.3% and 6.3 phantom reps for a naive counter. An ablation shows that 3D world landmarks matter most (frontal-view MAE 0.051 vs. 0.778 without them). We release the benchmark and a protocol for a real-video study.**

***Index Terms*—human pose estimation, repetition counting, smart TV, fitness, multi-device interaction, large language models**

I. Introduction

Home workouts on a TV usually mean choosing from a library of fixed videos. Nobody adjusts the session to the person, counts their reps or notices that a squat is too shallow. Camera-based coaching exists on phones and laptops [1], [2], but the TV, the screen people actually exercise in front of, rarely has a camera, and TV platforms such as Vega OS offer apps no audio-capture or speech-recognition API.

Fire-Fit addresses this with a split design: *the TV runs the workout, the phone is its eyes and ears*. The TV shows the session (an animated demonstration figure, countdowns, form tips, calories), while a phone propped two to three metres away runs pose estimation and speech recognition in its browser. Pairing is by QR code, with nothing to install. The same counter runs directly in a laptop browser when a webcam is present.

This paper makes four contributions:

1) A **phone-as-sensor architecture** for camera-less TVs that keeps video on the phone and sends only discrete events (Section III).
2) A **rule-based rep counter** for 19 exercise families with eight switchable robustness mechanisms, small enough to run identically on the TV, phone and laptop (Section IV).
3) A **synthetic benchmark and ablation** that generates labelled pose sequences from the app's own exercise animations under realistic camera degradations (Section V).
4) An open **real-video evaluation protocol** and tooling (Section VI).

II. Related Work

Pose estimation. OpenPose [3] made multi-person 2D pose estimation practical; BlazePose [4], distributed through MediaPipe [5], runs on phones in real time and also estimates metric 3D "world" landmarks. Fire-Fit uses the MediaPipe Pose Landmarker in the browser.

Repetition counting. Learned counters such as RepNet [6] and TransRAC [7] count repetitions of arbitrary actions from video; earlier work counted live from motion periodicity [8]. These methods are class-agnostic but heavy for a phone browser and do not explain *why* a rep did not count. GymCam [9] tracks exercises with a fixed camera in a gym. Rule-based counters on pose keypoints, as in Pose Trainer [1], are interpretable and cheap but fragile; this paper studies which mechanisms make such a counter robust. We adopt RepNet's MAE and OBOA metrics.

Form feedback. AIFit [2] learns interpretable 3D feedback models. Fire-Fit uses hand-written per-family checks (squat depth, hip sag, lockout) because they can be explained in one spoken sentence.

III. System Design

Fig. 1 shows the components.

TV app. One React Native codebase targets Vega OS and Fire OS (and the web). Screens are laid out at 1920×1080 and every dimension is converted to device units before rendering, so text stays sharp on 720p–4K panels, including Android TV devices that report 960×540 dp. The user drives everything with the remote's D-pad or by voice. Each of the 32 moves

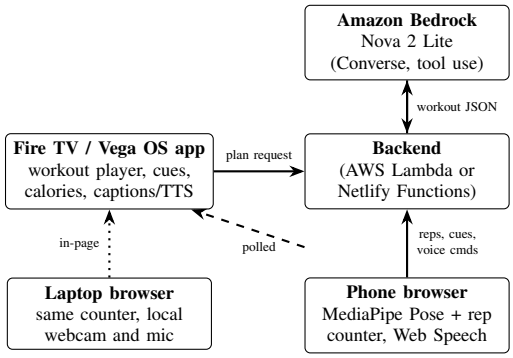


Fig. 1. Fire-Fit architecture. Video stays on the phone or laptop; only events reach the TV.

has a hand-built animated stick figure; the same animations drive the benchmark in Section V.

Workout generation. The user picks a goal, duration, level, equipment and limits (e.g., "knee friendly"). The backend calls Amazon Nova 2 Lite through the Bedrock Converse API [10] with a forced `emit_workout` tool whose JSON schema describes warm-up, main and cool-down segments. Every field is validated and clamped before the TV sees it, and a deterministic generator is used if the model is unavailable, so the TV never dead-ends.

Phone companion. Scanning the TV's QR code opens a web page bound to a four-digit session. The page runs the Pose Landmarker (full model, lite fallback) at up to 30 fps and the Web Speech API [11]. It posts rep events, form cues and parsed commands (*start, wait, skip, rest, slower, faster, easier, end*) to the session; the TV polls it and speaks the cue. Sessions are stored in DynamoDB on AWS or in Netlify Blobs. A laptop with a webcam runs the same code inside the TV web app with no pairing; the mode is detected automatically.

Calories. Energy is estimated from the Mifflin–St Jeor resting metabolic rate [12] and each move's MET value [13]: $\dot{E} = \mathrm{MET} \cdot \mathrm{BMR}/86400$ kcal s^{-1}.

Privacy. No frame leaves the phone or laptop. The backend sees only rep counts, short cue strings and command words; body data for calories stays on the TV.

IV. Rep Counter

Each exercise maps to one of 23 *families*, 19 of which count reps (squat, lunge, push-up, hinge, curl, press, row, bridge, crunch, burpee, jumping jacks, high knees, climbers, skater, boxing, circles, marching, band pull-apart, halo). A family defines a scalar metric per frame, usually a joint angle (knee angle for squats, elbow angle for push-ups) or a normalised distance (feet width over torso length for jacks), plus form checks.

Hysteresis counting. For a "down" movement with thresholds $\ell < h$, a rep is counted when the metric goes below ℓ and then back above h (e.g., squat knee angle $\ell = 130°$, $h = 160°$). The extreme value of the rep feeds the form check.

Robustness mechanisms. Each can be disabled for ablation:

- *3D world landmarks*: angles from metric 3D landmarks, so a squat seen from the front still shows knee flexion.
- *Aspect correction*: without 3D data, image x is scaled by the frame's aspect ratio before computing angles (portrait phones).
- *Frame gating*: frames are ignored if a needed joint has visibility < 0.65 or jumps more than 0.6 torso lengths between frames (a guessed, occluded joint).
- *Posture gate*: each family has an expected torso posture (upright, squatting, lying, prone); curls do not count as push-ups.
- *Sticky side*: in side view the more visible side is used and kept, so the counter does not flip between left and right on flicker.
- *Start rule*: the counter arms only after the start position is seen, so walking into frame mid-squat is not a rep.
- *Minimum duration*: a rep faster than a family-specific minimum (e.g., 500 ms for squats, 300 ms for jacks) is rejected, and stage-based families must hold a stage for 150 ms.
- *Alternation*: alternating moves (marching, high knees, climbers) count a leg only after the other side.

Landmarks are smoothed with a One Euro filter [14]. A half-finished rep is dropped after 8 s or when the person is lost from view.

The counter is about 700 lines of TypeScript with no dependencies. The same file runs on the phone, laptop and TV; a test asserts the copies are identical. It costs about 30 µs per frame (filter and rules) in Node.js 22 on a 2.1 GHz Xeon core, negligible against the 33 ms frame budget.

V. Synthetic Evaluation

Benchmark. Real labelled footage is expensive, so we first built a controlled benchmark from the app's 23 counted exercise animations. Each animation is played for six cycles at 30 fps and converted to the 33 BlazePose landmarks, giving a known true count (bilateral moves count each side). Six conditions model camera input: *clean*; *noise* (Gaussian landmark noise, $\sigma = 0.012$ of frame size); *occlusion* (12% of frames have one limb joint displaced by up to 0.2 with low visibility); *frontal* (side-view moves rotated to face the camera, 18 exercises); *portrait* (aspect 0.75, no 3D data); and *flicker* (far-side joints jitter and alternate in visibility). Noise and occlusion use three seeds. We add 21 distractor sequences with true count zero: 5 Hz shaking in four exercises' poses, and the wrong exercise (curls when push-ups are expected, marching for crunches, squats for bridges). In total there are 246 sequences.

Metrics. Following RepNet [6], MAE $= \frac{1}{N}\sum |\hat{c} - c|/c$ and OBOA is the share of sequences with $|\hat{c} - c| \le 1$. Phantom reps are the mean count on distractors. Averages are macro-averages over the six conditions.

Configurations. The full system; the full system minus each mechanism; minus the One Euro filter; and a baseline with everything off.

TABLE I
MAE / OBOA BY CAMERA CONDITION

	Clean	Noise	Occlusion	Frontal	Portrait	Flicker
Full system	0.040 / 96%	0.144 / 83%	0.064 / 88%	0.051 / 94%	0.040 / 96%	0.040 / 96%
Baseline (all off)	0.004 / 100%	0.103 / 87%	0.034 / 94%	0.713 / 28%	0.105 / 91%	0.312 / 70%

TABLE II
ABLATION ON THE SYNTHETIC BENCHMARK (MACRO-AVERAGE OVER SIX CONDITIONS)

Configuration	MAE ↓	OBOA ↑	Phantom ↓
Full system	0.063	92.1%	0.00
– 3D world landmarks	0.190	78.6%	0.00
– aspect correction	0.085	89.9%	0.00
– frame gating	0.063	92.1%	0.00
– posture gate	0.063	92.1%	0.00
– sticky side	0.111	87.7%	0.00
– start rule	0.036	96.9%	0.00
– min. duration	0.063	92.1%	5.00
– alternation	0.063	92.1%	0.00
– One Euro filter	0.065	91.3%	0.00
Baseline (all off)	0.212	78.3%	6.33

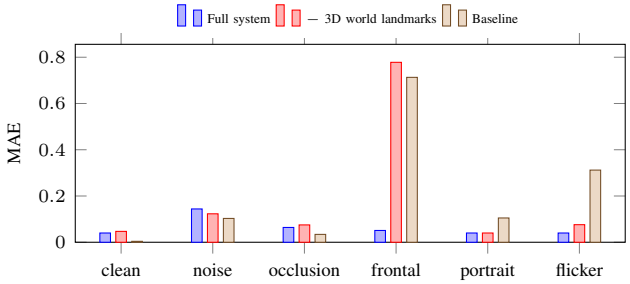


Fig. 2. MAE by condition (lower is better).

Results. Tables I and II and Fig. 2 show the results. The full system reaches MAE 0.063 and OBOA 92.1% with no phantom reps, against 0.212, 78.3% and 6.3 phantom reps for the baseline.

3D world landmarks matter most: without them frontal-view MAE rises from 0.051 to 0.778, because knee and hip flexion is nearly invisible in a 2D frontal projection. *Sticky side* prevents the flicker failure (MAE 0.040 vs. 0.330), *aspect correction* the portrait failure (0.040 vs. 0.170), and *minimum duration* removes all 10 phantom reps per shaking sequence.

Three findings were less flattering, and we report them as found. First, the *start rule* costs accuracy here (MAE 0.036 without it): seven animations begin mid-rep, and the rule deliberately discards that first partial rep. In real use this is the intended behaviour, but the benchmark penalises it. Second, under *noise* the full system is worse than the baseline (0.144 vs. 0.103): smoothing and gating trade lost reps for zero false positives. Small-amplitude moves suffer most; dumbbell and kettlebell halos lose every rep at $\sigma = 0.012$. Third, removing frame gating, the posture gate or alternation alone changes nothing on this benchmark, although the baseline counts 2.7 phantom reps on wrong-exercise sequences: these protections overlap, so only removing several exposes the failure. Real footage, with genuinely occluded joints, is needed to separate them.

Tuning. An earlier filter setting ($f_{c,\min} = 1.2$ Hz, $\beta = 0.5$) over-smoothed fast moves: with it, the full system missed every high-knee and mountain-climber rep and reached only MAE 0.183 (OBOA 82.7%). Raising β to 20 and $f_{c,\min}$ to 1.5 Hz restored them; adding the 150 ms stage hold for jacks then removed the phantom reps that faster filtering let through. The benchmark exposed both problems before users did.

VI. Real-Video Study

Synthetic sequences come from the same animations that the thresholds were designed against, so they measure robustness to degradations, not accuracy on people. We therefore define a real-video protocol and release the tooling: participants record 8–15 reps of eight exercises from side and front views with a phone (portrait, 2–3 m away) and a laptop webcam, plus distractor clips. Two annotators count each clip independently and resolve disagreements together. Landmarks are extracted offline with the same Pose Landmarker model, and the same ablations are scored.

[Real-video results pending: record the clips (backend/eval/real/README.md), run the scorer, and rebuild; the table is inserted automatically.]

VII. Limitations

The synthetic benchmark shares its source with the counter's design, and its degradations are simplified models of real camera error. The counter handles one person and 19 counted families; new exercises need new rules, unlike class-agnostic counters such as RepNet [6]. Form checks are heuristics and have not been validated against expert ratings. Calorie estimates use population equations and are approximate.

VIII. Conclusion

Fire-Fit shows that a camera-less TV can still watch a workout by borrowing the viewer's phone, without sending video anywhere. A small, interpretable counter can be made robust with a handful of mechanisms, chiefly 3D world landmarks, and a synthetic benchmark built from the app's own animations is a cheap way to find regressions before users do. Future work is the real-video study, learned per-family thresholds, and multi-person tracking.

Code and Data Availability

[Add repository URL.] The benchmark is reproduced with `npx tsx eval/synthetic.ts`; the real-video tooling is in `eval/real/`.

Acknowledgment

In line with IEEE policy on AI-generated content, we disclose that an AI assistant (Anthropic's Claude) was used to help write the application code, the evaluation scripts and a draft of this paper. All results come from running the released code; the author reviewed and is responsible for all content. [Edit to reflect exactly how you used AI tools.]

References

[1] S. Chen and R. R. Yang, "Pose trainer: Correcting exercise posture using pose estimation," arXiv:2006.11718, 2020.

[2] M. Fieraru, M. Zanfir, S.-C. Pirlea, V. Olaru, and C. Sminchisescu, "AIFit: Automatic 3D human-interpretable feedback models for fitness training," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9919–9928.

[3] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, 2021.

[4] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.

[5] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[6] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman, "Counting out time: Class agnostic video repetition counting in the wild," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 387–10 396.

[7] H. Hu, S. Dong, Y. Zhao, D. Lian, Z. Li, and S. Gao, "TransRAC: Encoding multi-scale temporal correlation with transformers for repetitive action counting," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 19 013–19 022.

[8] O. Levy and L. Wolf, "Live repetition counting," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 3020–3028.

[9] R. Khurana, K. Ahuja, Z. Yu, J. Mankoff, C. Harrison, and M. Goel, "GymCam: Detecting, recognizing and tracking simultaneous exercises in unconstrained scenes," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, no. 4, pp. 185:1–185:17, 2018.

[10] Amazon Web Services, "Carry out a conversation with the Converse API operations (Amazon Bedrock user guide)," https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html, accessed Oct. 2026.

[11] W3C Audio Community Group, "Web speech API," https://webaudio.github.io/web-speech-api/, accessed Oct. 2026.

[12] M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty, and Y. O. Koh, "A new predictive equation for resting energy expenditure in healthy individuals," *Am. J. Clin. Nutr.*, vol. 51, no. 2, pp. 241–247, 1990.

[13] B. E. Ainsworth, W. L. Haskell, S. D. Herrmann, N. Meckes, D. R. Bassett, C. Tudor-Locke, J. L. Greer, J. Vezina, M. C. Whitt-Glover, and A. S. Leon, "2011 compendium of physical activities: A second update of codes and MET values," *Med. Sci. Sports Exerc.*, vol. 43, no. 8, pp. 1575–1581, 2011.

[14] G. Casiez, N. Roussel, and D. Vogel, "1€ filter: A simple speed-based low-pass filter for noisy input in interactive systems," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, 2012, pp. 2527–2530.